

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

```
df = pd.read_csv("Zomato-data-.csv")
df.columns = df.columns.str.strip()
```

```
df.columns = df.columns.str.strip()
```

```
# Clean cost
df["approx_cost(for two people)"] = df["approx_cost(for two people)"].astype(str)
df["approx_cost(for two people)"] = df["approx_cost(for two people)"].str.replace(",", "")
df["approx_cost(for two people)"] = pd.to_numeric(
    df["approx_cost(for two people)"], errors="coerce"
)
```

```
# Clean rate
df["rate"] = df["rate"].astype(str).str.split("/").str[0]
df["rate"] = pd.to_numeric(df["rate"], errors="coerce")
```

```
# Drop NaNs
df = df.dropna(subset=["approx_cost(for two people)", "rate", "votes"])
# Drop NaNs
df = df.dropna(subset=["approx_cost(for two people)", "rate", "votes"])
```

```
features = ["rate", "votes"]

X = df[features]
y = df["approx_cost(for two people)"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
r2_lr = r2_score(y_test, y_pred_lr)
```

```
rf = RandomForestRegressor(
    n_estimators=100,
    max_depth=10,
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2_rf = r2_score(y_test, y_pred_rf)
```

```
print("Linear Regression:")
print("RMSE:", rmse_lr)
print("R2 Score:", r2_lr)

print("\nRandom Forest:")
print("RMSE:", rmse_rf)
print("R2 Score:", r2_rf)
```

```
Linear Regression:
RMSE: 200.84427401711983
R2 Score: 0.011043852767160911
```

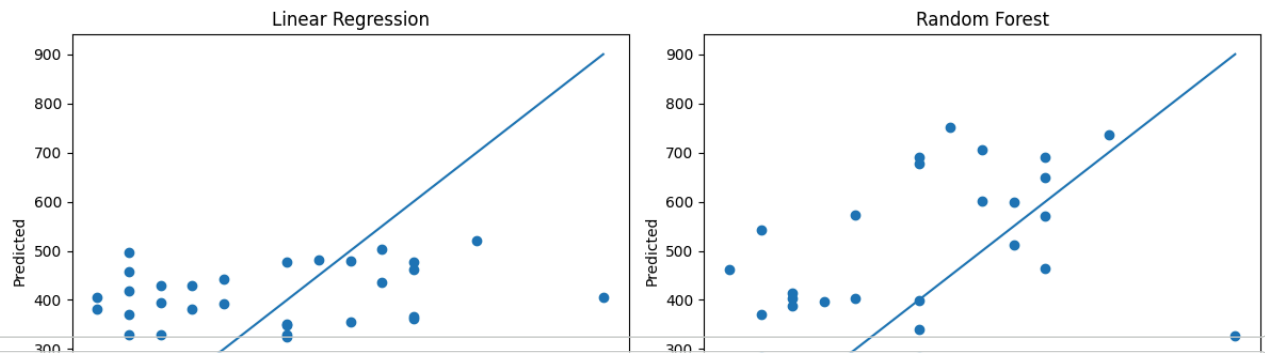
Random Forest:
 RMSE: 207.71717237775002
 R2 Score: -0.057798456293846945

```
plt.figure(figsize=(12, 5))

# Linear Regression Plot
plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred_lr)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()])
plt.title("Linear Regression")
plt.xlabel("Actual")
plt.ylabel("Predicted")

# Random Forest Plot
plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_rf)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()])
plt.title("Random Forest")
plt.xlabel("Actual")
plt.ylabel("Predicted")

plt.tight_layout()
plt.show()
```



```
# Create comparison dataframe
comparison_df = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted_LR": y_pred_lr,
    "Predicted_RF": y_pred_rf
})

# Calculate absolute error
comparison_df["Error_LR"] = abs(comparison_df["Actual"] - comparison_df["Predicted_LR"])
comparison_df["Error_RF"] = abs(comparison_df["Actual"] - comparison_df["Predicted_RF"])

# Sort by smallest error (best predictions)
comparison_df = comparison_df.sort_values(by="Error_RF")

# Show top 5 best predictions
print(comparison_df.head(5))
```

	Actual	Predicted_LR	Predicted_RF	Error_LR	Error_RF
3	400	350.714414	398.625000	49.285586	1.375000
19	600	461.700173	570.127282	138.299827	29.872718
13	700	519.725437	736.294872	180.274563	36.294872
9	550	503.822084	511.532967	46.177916	38.467033
12	150	371.088321	196.432992	221.088321	46.432992

Start coding or [generate](#) with AI.

